Metric/Nonmetric Sammon Mapping

Jan de Leeuw

October 28, 2026

We present R and C implementations of the majorization method for metric and non-metric (ordinal) versions of Sammon mapping, the multidimensional scaling technique proposed by John W. Sammon Jr. in 1969. The non-metric version is new, as far as I know. It requires a special purpose monotone regression technique. The R and C versions are compared for speed, with the C version anywhere from 10 to 100 times as fast as the R version.

Table of contents

1 Introduction **2**

2 Properties of Loss **2**

3 Algorithm **3**

4 Software **6**

5 Examples **7**

5.1 Results 7

5.2 Timing 7

5.3 Shepard Plots 8

References **13**

Note: This is a working manuscript which will be expanded/updated frequently. All suggestions for improvement are welcome. All Rmd, tex, html, pdf, R, and C files are in the public domain. Attribution will be appreciated, but is not required. The files can be found at https://github.com/deleeuw/sammon

1 Introduction

In engineering and computer science *Sammon Mapping* is a popular multidimensional scaling (MDS) method. The technique was introduced in Sammon Jr (1969). It was originally intended to map points from a higher-dimensional Euclidean space into points in a lower-dimensional Euclidean space by approximating the given higher dimensional distances by best-fitting lower dimensional ones. The *MASS* package for R implemented the *sammon()* function, which generalizes the original idea by allowing the higher-dimensional distances to be replaced by any positive symmetric matrix of dissimilarities. This was again generalized in the packages *stops* (Rusch et al. (2023)) and *smacofx* (Rusch et al. (2025)) where optimization is over low-dimensional configations and over power transforms of the dissimilarities.

The Sammon loss function is

$$\sigma(X,\Delta)=\frac{1}{\sum\sum_{1\leq i<j\leq n}w_{ij}\delta_{ij}}\sum\sum_{1\leq i<j\leq n}w_{ij}\frac{(\delta_{ij}-d_{ij}(X))^2}{\delta_{ij}}. \tag{1}$$

The metric MDS problem we address in this paper is minimizing (1) over the $n\times p$ *configurations* X. In the non-metric case we minimize in addition over the symmetric non-negative and hollow matrices $\Delta:=\{\delta_{ij}\}$, where the δ_{ij} are monotone with the given dissimilarities.

2 Properties of Loss

The usual stress loss function in smacof, due originally to Kruskal (1964a) and Kruskal (1964b), is

$$\sigma(X,\Delta)=\frac{\sum\sum_{1\leq i<j\leq n}w_{ij}(\delta_{ij}-d_{ij}(X))^2}{\sum\sum_{1\leq i<j\leq n}w_{ij}\delta_{ij}^2}. \tag{2}$$

If we compare (1) and (2) we see that in Kruskal's stress the sum of the squared residuals is normalized by dividing by the sum of the squared dissimilarities. This means minimum stress is between zero and one, and it also prevents us from considering the trivial zero-residual solution $X=0$ and $\Delta=0$. In Sammon loss each residual is normalized separately, based on the idea that large dissimilarities are somehow less reliable or stable and should be downweighted.

If δ_{ij} and $d_{ij}(X)$ are close then

$$\sqrt{d_{ij}(X)}-\sqrt{\delta_{ij}}\approx\frac{1}{2}\frac{1}{\sqrt{\delta_{ij}}}(d_{ij}(X)-\delta_{ij}), \tag{3}$$

and thus

$$\sum w_{ij}\frac{(\delta_{ij}-d_{ij}(X))^2}{\delta_{ij}}\approx 4\sum w_{ij}(\sqrt{\delta_{ij}}-\sqrt{d_{ij}(X)})^2 \tag{4}$$

The loss function on the right-hand side of (4), which we will informally call *sqrt-stress*, is the special case of *rStress* with $r=\frac{1}{4}$ (de Leeuw et al. (2016)).

3 Algorithm

In metric (ratio) MDS we minimize over X for fixed Δ. This is a standard smacof problem with weights $\tilde{w}_{ij}:=w_{ij}\delta_{ij}^{-1}$. We use majorization steps to decrease loss. This is identical to the *sammon* option in *smacofx*. Because the δ_{ij} are the same in each iteration, the same is true for the weights $\tilde{w}_{ij}$. Without going into details

In the non-metric case we use block relaxation, i.e. we alternate majorization steps with minimizing over Δ, satisfying the ordinal constraints, for fixed X. As far as I know this has not been implemented before, and is non-standard. The reason …

First, use the fact that

$$\min_X \sigma(X,\Delta)=\min_X \sigma(X,\lambda\Delta), \tag{5}$$

i.e. the problem is homogeneous of degree zero in Δ. Thus we can require without loss of generality that

$$\sum_{1\leq i<j\leq n}\sum w_{ij}\delta_{ij}=1. \tag{6}$$

With this normalization we have

$$\sigma(X,\Delta)=1-2\sum_{1\leq i<j\leq n}\sum w_{ij}d_{ij}(X)+\sum_{1\leq i<j\leq n}\sum w_{ij}\frac{d_{ij}^2(X)}{\delta_{ij}}. \tag{7}$$

Thus minimizing ω for fixed X over non-negative, isotone and normalized Δ means minimizing the third term on the right. And this means minimizing a separable, differentiable, and strictly convex function over a polyhedral convex set.

To simplify notation we define the extended real valued function f equal to

$$f(x):=\sum_{i=1}^n w_i\frac{y_i}{x_i} \tag{8}$$

if $x_i>0$ for all i, and to $+\infty$ otherwise. Thus the effective domain $\text{dom}(f)$ is the interior of the positive orthant. In the context of Sammon Mapping the y_i are squared distances and the x_i are the transformed dissimilarities.

The *Sammon Monotone Regression* or SMR problem is defined as minimization of f over the intersection of the polyhedral convex cone $\mathcal{K}$ defined by $0 < x_1 \leq \cdots \leq x_n$ and the affine set $\mathcal{A}$ defined by $w'x = 1$. We assume that all w_i and all y_i are positive, and that the w_i add up to one.

The first and second partials of f are

$$\mathcal{D}_i f(x) = -w_i \frac{y_i}{x_i^2}, \tag{9}$$

$$\mathcal{D}_{ii} f(x) = 2w_i \frac{y_i}{x_i^3}, \tag{10}$$

and $\mathcal{D}_{ik} f(x) = 0$ if $i \neq k$. This shows f is convex and twice differentiable on $\mathbf{dom}(f)$, and thus on the intersection with $\mathcal{K} \cap \mathcal{A}$.

First some preliminary results.

Lemma 3.1.

$$\min_{x \in \mathcal{K} \cap \mathcal{A}} f(x) \leq w'y \tag{11}$$

Proof. The vector x with $x_i = 1$ for all i is in $\mathcal{K} \cap \mathcal{A}$. □

It follows from Lemma 3.1 that we can add the constraint $f(x) \leq w'y$ to the minimization problem and still have the same minimum and minimizer.

Lemma 3.2. *Suppose* $y_1 \leq \cdots \leq y_n$. *Then*

$$\overline{x}_i = \frac{\sqrt{y_i}}{\sum_{k=1}^n w_k \sqrt{y_k}}. \tag{12}$$

Proof. The necessary conditions for a minimum of f on $\mathcal{A}$ are

$$-w_i \frac{y_i}{x_i^2} = \lambda w_i, \tag{13}$$

for all i, together with the side condition $w'x = 1$. From (13) it follows that the solution $\overline{x}$ must be proportional to $\sqrt{y}$, where the square root can have either sign. Of the 2^n solutions only one is in the effective domain of f, the one for which all all square roots are taken with a positive sign. This solution is also in $\mathcal{K}$. Applying the side condition gives (12). □

The next rather trivial lemma deals with the case $n = 1$, in which w, x and y are one-element vectors, which identify with the corresponding scalars.

Lemma 3.3. *If $n = 1$ then the minimizer $\overline{x}$ is equal to one and the minimum is y.*

Proof. w adds up to one, so $w = 1$. Also wx must be one, so $\overline{x} = 1$. □

The following theorem is of prime importance, because it shows the SMR problem can be solved with a variation of the Pool Adjacent Violaters Algorithm (PAVA). For the details on PAVA, see for example de Leeuw et al. (2009).

Theorem 3.1. *Suppose $\overline{x}$ is the optimum solution. If $y_i \geq y_{i+1}$ then $\overline{x}_i = \overline{x}_{i+1}$.*

Proof. We show that $y_i \geq y_{i+1}$ and $\overline{x}_i < \overline{x}_{i+1}$ leads to a contradiction. A necessary and sufficient condition for $\overline{x}$ to be the optimum solution is

$$(x - \overline{x})'\mathcal{D}f(\overline{x}) = -\sum_{i=1}^n w_i \frac{y_i}{\overline{x}_i^2}(x_i - \overline{x}_i) = f(\overline{x}) - \sum_{i=1}^n w_i \frac{y_i}{\overline{x}_i^2} x_i \geq 0 \tag{14}$$

for all $x \in \mathcal{K} \cap \mathcal{A}$ (Hiriart-Urruty and Lemaréchal (1993), Theorem 1.1.1, page 293).

Now suppose $\overline{x}_i < \overline{x}_{i+1}$. Then for $\epsilon > 0$ small enough

$$z = \overline{x} + \epsilon \left(\frac{e_i}{w_i} - \frac{e_{i+1}}{w_{i+1}} \right) \tag{15}$$

is also in $\mathcal{K} \cap \mathcal{A}$. In (15) we add a small amount to $\overline{x}_i$ and subtract a small amount from $\overline{x}_{i+1}$, while leaving all other elements of $\overline{x}$ unperturbed. Since

$$\sum_{i=1}^n w_i \frac{y_i}{\overline{x}_i^2} z_i = f(\overline{x}) + \epsilon \left(\frac{y_i}{\overline{x}_i^2} - \frac{y_{i+1}}{\overline{x}_{i+1}^2} \right) \tag{16}$$

we must have

$$\frac{y_i}{\overline{x}_i^2} \leq \frac{y_{i+1}}{\overline{x}_{i+1}^2} \tag{17}$$

But $y_i \geq y_{i+1}$ and $\overline{x}_i < \overline{x}_{i+1}$ implies

$$\frac{y_i}{\overline{x}_i^2} > \frac{y_{i+1}}{\overline{x}_{i+1}^2} \tag{18}$$

and thus $\overline{x}$ cannot be the optimal solution. □

Theorem 3.1 can be used to replace an SMR problem of size n with an SMR problem of size $n - 1$. If $y_i \geq y_{i+1}$ we remove these two y-values and put the single value

$$\tilde{y}_i := \frac{w_i y_i + w_{i+1} y_{i+1}}{w_i + w_{i+1}} \tag{19}$$

in their place. This new value gets the weight $\tilde{w}_i := w_i + w_{i+1}$.

Theorem 3.2. *Suppose $\hat{x}$ solves the monotone regression problem for the weighted least squares norm*

$$\hat{x} := \underset{x_1 \leq \cdots \leq x_n}{\text{argmin}} \ (x-y)'W(x-y) \tag{20}$$

then

$$\overline{x}_i = \frac{\sqrt{\hat{x}_i}}{\sum_{k=1}^n w_k \sqrt{\hat{x}_k}} \tag{21}$$

solves the SMR problem.

Proof. We use Theorem 3.1 repeatedly until we have an SMR problem of size r for which $\tilde{y}_1 < \cdots < \tilde{y}_r$. We can use the same sequence of pooling adjacent violators as in least squares monotone regression, and we find the same weighted average pooled values in each step. When we have reduced the problem to a strictly increasing $\tilde{y}$ sequence we apply Lemma 3.2 (and if $r = 1$ we apply Lemma 3.3). We then expand each block again to a length equal to the number of averaged elements. □

A small example illustrates our PAVA variation. Start with y equal to $(1, 2, 3, 1, 2, 3, 5, 1)$ and w equal to $(1, 1, 1, 1, 1, 1, 1, 1)$. Merge elements 3 and 4 to get $y = (1, 2, 2, 2, 3, 5, 1)$ and $w = (1, 1, 2, 1, 1, 1, 1)$. Now merge elements 2, 3, and 4 to get $y = (1, 2, 3, 5, 1)$ and $w = (1, 4, 1, 1, 1)$. Merge 4 and 5 to get $y = (1, 2, 3, 3)$ and $w = (1, 4, 1, 2)$ and finally merge 3 and 4 to get $y = (1, 2, 3)$ and $w = (1, 4, 3)$. Thus

$$\overline{x} = \frac{1}{1 + 4\sqrt{2} + 3\sqrt{3}}(1, \sqrt{2}, \sqrt{2}, \sqrt{2}, \sqrt{2}, \sqrt{3}, \sqrt{3}, \sqrt{3})$$

Remember that in the context of Sammon Mapping the least squares monotone regression is on the squared distances.

4 Software

The github repository contains R and C versions of the smacofSSSammon program. Both smacofSSSammonC() and smacofSSSammonR() have the same default values of the parameters and the two programs are structured the same way. We iterate a maximum of 1000 iterations until the change in stress from one iteration to the next is less than 10^{-6}.

A single majorization update is alternated with a single monotone regression. Note that the two steps in an iteration are different. In majorization we go one step towards minimizing σ over X for fixed Δ, in the monotone regression we actually minimize σ over Δ for given X.

smacofSSSammonC() has a driver in R that sets up the MDS data structure (de Leeuw (2025)). Both programs use a number of R utilities for data manipulation and to compute the initial configuration. One important difference is that the R version uses gpava(), which is written in R, for monotone regression (de Leeuw et al. (2009)), while the R version uses the C routine monotone() from Busing (2022).

5 Examples

5.1 Results

We use two classical MDS data sets, both rather small. The Ekman data (Ekman (1954)) are dissimilarities between 14 colors, the Morse data (Rothkopf (1957)) have 37 Morse code signals. We are only interested in the performance of the software in this paper, not in interpreting the resulting MDS configurations. Only the results for C version are given, because the R version gives exactly the same results.

For Ekman numerical Sammon mapping takes 21 iterations to arrive at stress 0.0222606801. For ordinal we need 31 iterations for stress 0.0006660664. Since the ordinal fit is very good the sqrt-stress of (4) should be close to Sammon stress. It is 0.0006701850.

For Morse numerical Sammon mapping takes 99 iterations to arrive at stress 0.0977737, for ordinal we need 92 iterations for stress 0.0398178. Because the fit is much worse than in the Ekman case we do not expect sqrt-stress to be that close to Sammon stress. For the ordinal case final sqrt-stress is 0.0445148.

5.2 Timing

The results of a comparison of the R and C implementations of Sammon mapping with microbenchmark (Mersmann (2024)) yields the times in microseconds in the following tables.

```
Unit: microseconds
                                          expr       min         lq       mean
 smacofSSSammonR(ekmanData, ordinal = FALSE)  1703.468  1811.4005  1933.3255
 smacofSSSammonC(ekmanData, ordinal = FALSE)   324.884   336.0155   350.6931
  smacofSSSammonR(ekmanData, ordinal = TRUE) 11206.243 11632.0485 12516.6104
  smacofSSSammonC(ekmanData, ordinal = TRUE)   475.518   502.1680   529.7426
     median         uq       max neval
  1829.0715  1851.6215  3193.695   100
```

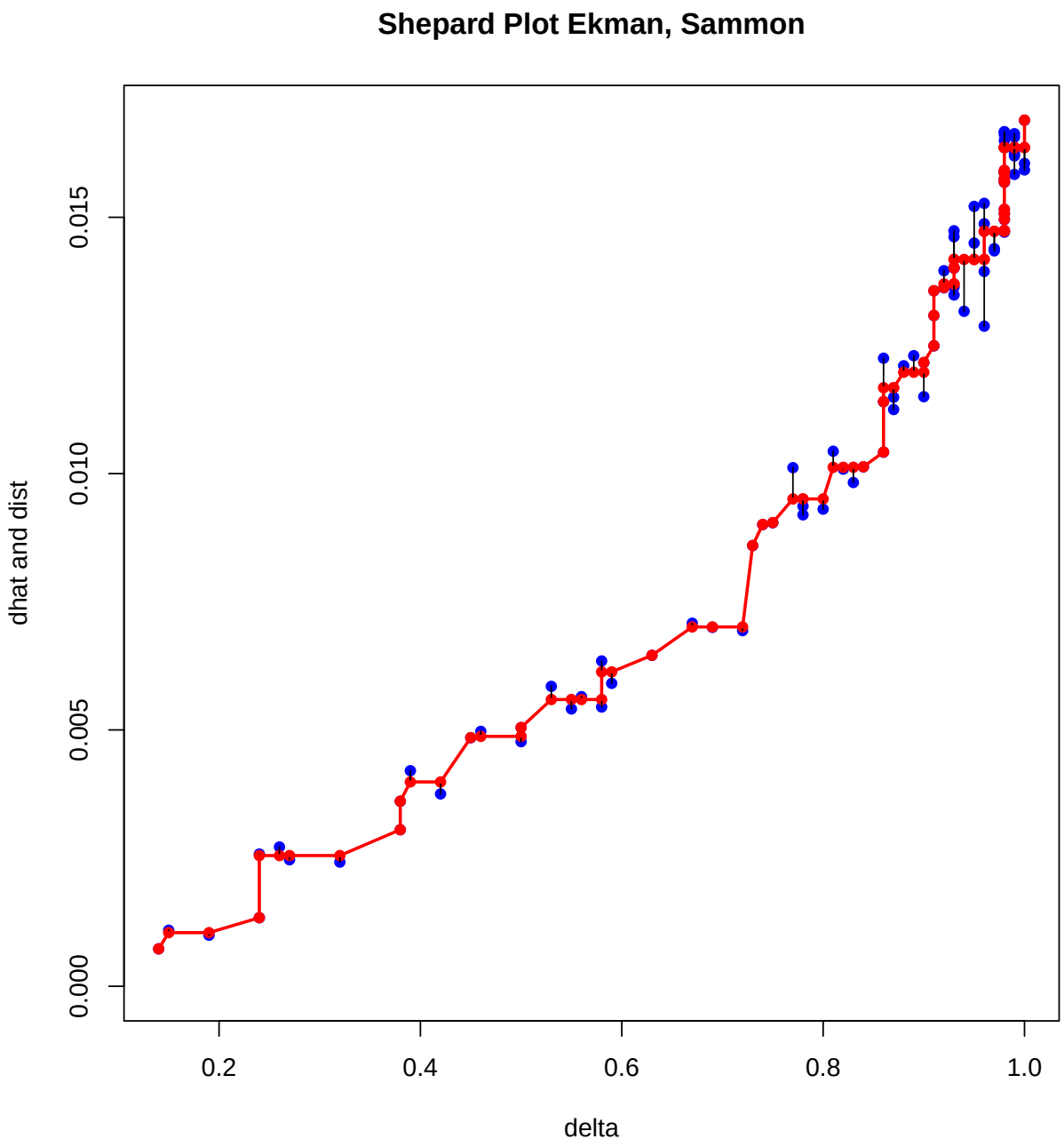
```
   343.4980   358.8730   438.495   100
 11817.6965 13016.3930 37433.656   100
   509.6915   526.7475  1768.617   100
```

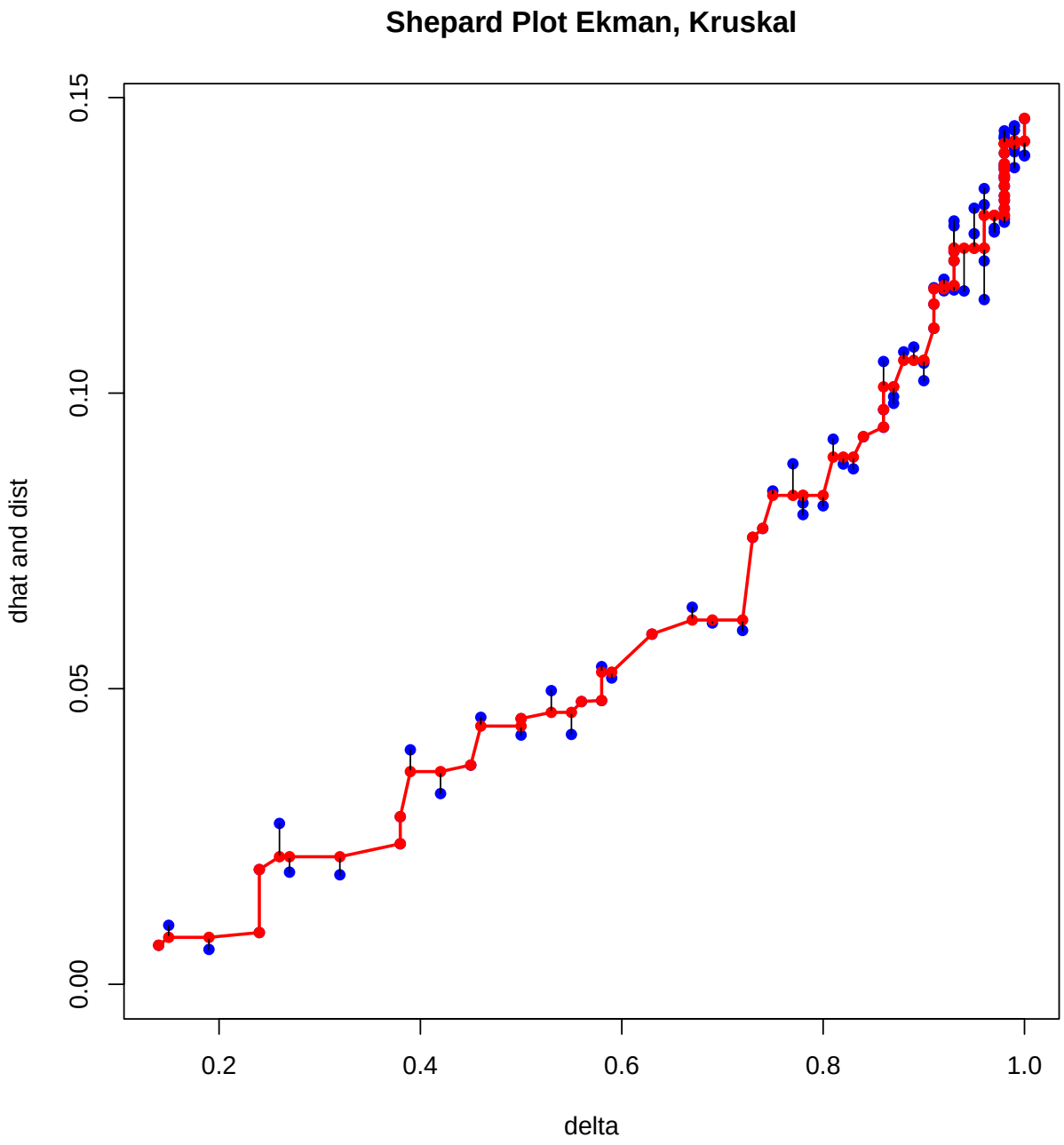
```
Unit: milliseconds
                                        expr        min         lq       mean
 smacofSSSammonR(morseData, ordinal = FALSE)  42.752791  44.653448  46.970267
 smacofSSSammonC(morseData, ordinal = FALSE)   3.714928   3.849921   3.982863
  smacofSSSammonR(morseData, ordinal = TRUE) 422.080035 429.738794 441.064172
  smacofSSSammonC(morseData, ordinal = TRUE)   4.679043   4.833531   4.972333
     median         uq        max neval
  45.320047  46.787949  93.754618   100
   3.916710   3.985241   6.546224   100
 432.782695 438.097115 483.844977   100
   4.900258   5.000011  10.259512   100
```

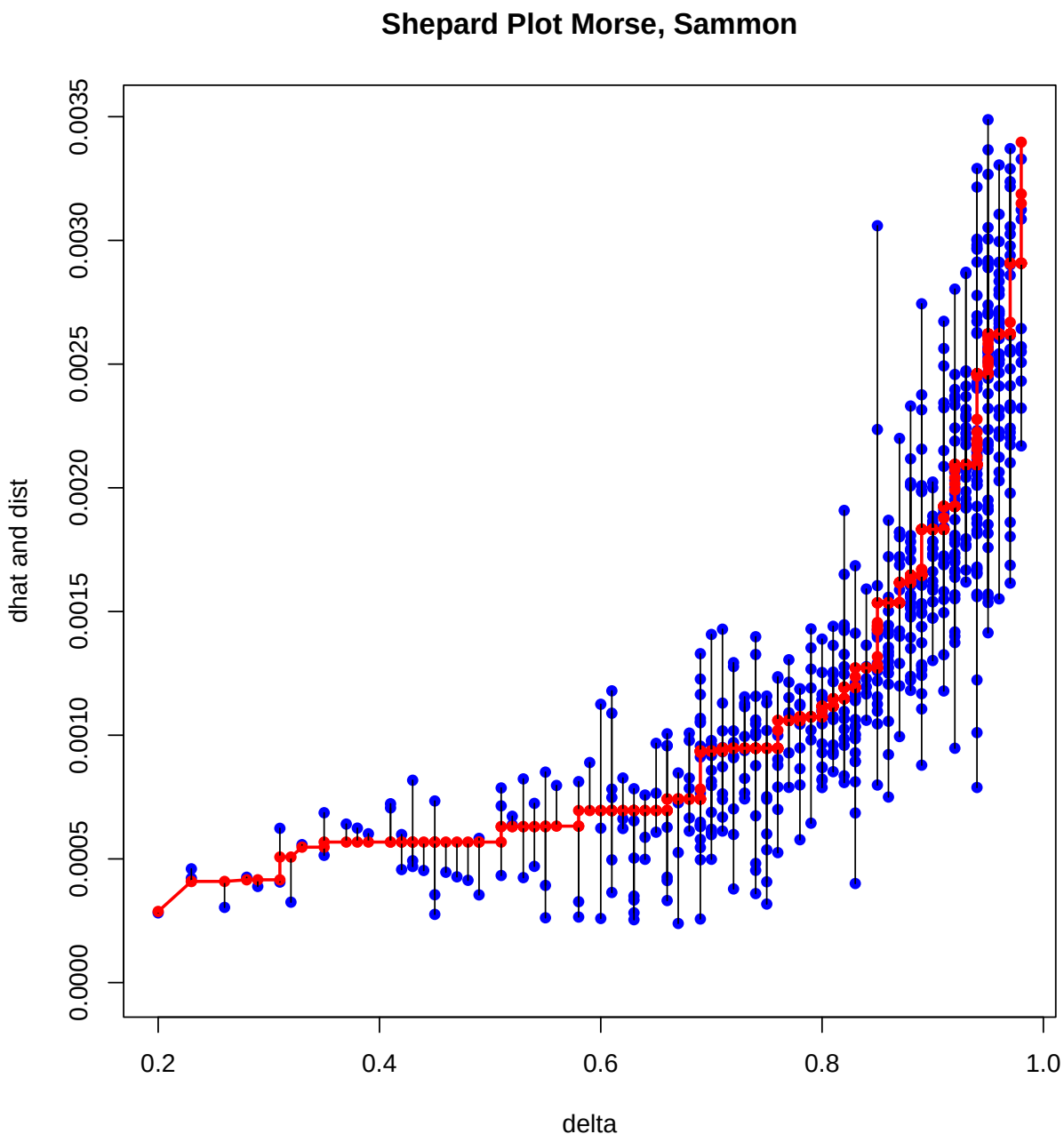
Of course these microbenchmark results depend on the default parameters of the programs (same in R and C), on the speed of my computer (Mac Mini with Apple M4 Pro, 64 GB of RAM, Golden Gate 27.2, Apple clang 21.0.0), and on my programming habits (same in R and C).

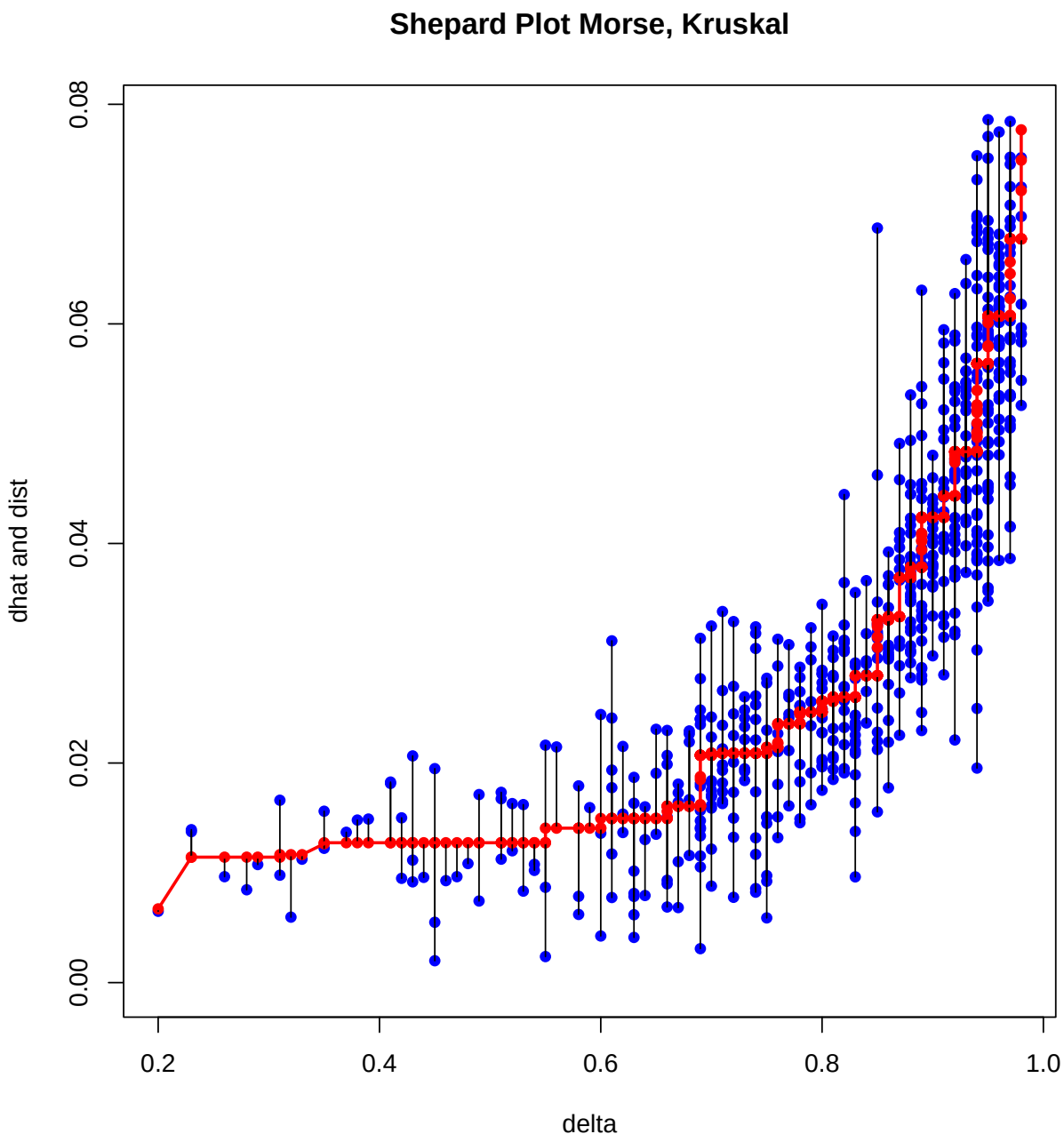
5.3 Shepard Plots

We use Shepard plots (de Leeuw and Mair (2015)) to illustrate the difference between the solutions for classical Kruskal stress (2) and Sammon stress (1). Both for Ekman and Morse we give the classical and Sammon solutions for the ordinal case, with the primary approach to ties.


Shepard Plot Ekman, Sammon
dhat and dist
0.000
0.005
0.010
0.015
0.2
0.4
0.6
0.8
1.0
delta


Shepard Plot Ekman, Kruskal
dhat and dist
0.00
0.05
0.10
0.15
0.2
0.4
0.6
0.8
1.0
delta


Shepard Plot Morse, Sammon
dhat and dist
0.0000
0.0005
0.0010
0.0015
0.0020
0.0025
0.0030
0.0035
0.2
0.4
0.6
0.8
1.0
delta



The examples show that the solutions are pretty similar, but definitely sammon MDS gives a better fit for the smaller dissimilarities and deemphasizes the larger ones. In the Ekman case the smallest dissimilarities are fitted almost perfectly

References

Busing, Frank M. T. A. 2022. "Monotone Regression: A Simple and Fast O(n) PAVA Implementation." *Journal of Statistical Software* 102 (Code Snippet 1). https://www.jstatsoft.org/index.php/jss/article/view/v102c01/4306.

de Leeuw, Jan. 2025. "Yet Another Smacof - Square Symmetric Case." https://deleeuw.github.io/janspubs/deleeuw-e-25-b/deleeuw-e-25-b.pdf.

de Leeuw, Jan, Patrick J. F. Groenen, and Patrick Mair. 2016. "Minimizing rStress Using Majorization." https://deleeuw.github.io/janspubs/deleeuw-groenen-mair-e-16-a/deleeuw-groenen-mair-e-16-a.pdf.

de Leeuw, Jan, Kurt Hornik, and Patrick Mair. 2009. "Isotone Optimization in R: Pool-Adjacent-Violators Algorithm (PAVA) and Active Set Methods." *Journal of Statistical Software* 32 (5): 1–24. https://deleeuw.github.io/janspubs/deleeuw-hornik-mair-a-09/deleeuw-hornik-mair-a-09.pdf.

de Leeuw, Jan, and Patrick Mair. 2015. "Shepard Diagram." In *Wiley StatsRef: Statistics Reference Online*. Wiley. https://deleeuw.github.io/janspubs/deleeuw-mair-c-15/deleeuw-mair-c-15.pdf.

Ekman, Gosta. 1954. "Dimensions of Color Vision." *Journal of Psychology* 38: 467–74.

Hiriart-Urruty, Jean-Baptiste, and Claude Lemaréchal. 1993. *Convex Analysiis and Minimization Algorithms Volume 1: Fundamentals*. Springer.

Kruskal, Joseph B. 1964a. "Multidimensional Scaling by Optimizing Goodness of Fit to a Nonmetric Hypothesis." *Psychometrika* 29: 1–27.

Kruskal, Joseph B. 1964b. "Nonmetric Multidimensional Scaling: a Numerical Method." *Psychometrika* 29: 115–29.

Mersmann, Olaf. 2024. *microbenchmark: Accurate Timing Functions*. https://CRAN.R-project.org/package=microbenchmark.

Rothkopf, Ernst Z. 1957. "A Measure of Stimulus Similarity and Errors in some Paired-associate Learning." *Journal of Experimental Psychology* 53: 94–101.

Rusch, Thomas, Jan de Leeuw, Lisha Chen, and Patrick Mair. 2025. *Package "smacofx":*

Flexible Multidimensional Scaling and 'smacof' Extensions. https://deleeuw.github.io/janspubs/rusch-deleeuw-chen-mair-m-25/rusch-deleeuw-chen-mair-m-25.pdf.

Rusch, Thomas, Patrick Mair, and Kurt Hornik. 2023. "Structure-Based Hyperparameter Selection with Bayesian Optimization in Multidimensional Scaling." *Statistics and Computing* 33 (28): 1–18.

Sammon Jr, John W. 1969. "A Nonlinear Mapping for Data Structure Analysis." *IEEE Transacttions on Computers* C-18 (5).